
Programming Tools for Scientific Computing: Final Project

Implementation and Analysis of a 2D Brusselator Reaction-Diffusion Solver

by Hardik (2791472)

Date: Summer Semester 2026

1. Review: Prof. Dr.-Ing. Christian Hasse
Pascal Steffens M.Sc.
Raphael Strickling M.Sc.
Nicolas Eckel M.Sc.
Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Contents

1	Technical Background	1
1.1	C++ and the Eigen Library	1
1.2	Build Automation with CMake	2
1.3	Shell Scripting with Bash	2
1.4	Containerization with Docker	2
1.5	Version Control and CI/CD with Git	4
1.6	Modular Software Architecture	4
2	Project Execution	6
2.1	Solver Implementation in C++	6
2.1.1	Simulation Parameters and Data Structures	6
2.1.2	Spatial Discretization and Boundary Conditions	7
2.1.3	Combined State Vector and System Matrix	9
2.1.4	Explicit Time Integration (Runge-Kutta 4th Order)	10
2.1.5	Implicit Time Integration (Implicit Euler)	11
2.1.6	Derivation of the Jacobian Matrix	12
2.1.7	Automated Unit Testing	13
2.2	Challenges and Solutions	14
2.2.1	Sparse Matrix Indexing and Boundary Conditions	14
2.2.2	Convergence and Stiffness of the Newton Solver	14
2.2.3	Coupling of Multi-Species Concentrations	15
2.2.4	What Was Easy	15
2.2.5	Future Improvements and Alternative Approaches	15
2.3	Simulation Runs and Discussion	16
3	Results	17
3.1	Parameter Variation 1: Influence of C_b on Pattern Formation	17
3.2	Parameter Variation 2: Time Step Influence on Implicit Euler Convergence	19
3.3	Execution Time Comparison: Implicit Euler vs Explicit RK4	21
3.4	Solver Accuracy and Absolute Error	22

1 Technical Background

This section serves as a technical reference and theoretical overview of the key programming tools and software engineering technologies utilized to implement, build, automate, and verify the 2D Brusselator solver.

1.1 C++ and the Eigen Library

C++ is widely used in scientific computing due to its high execution performance, low-level memory control, and support for object-oriented and generic programming paradigms. This project is implemented using the C++17 standard, leveraging type safety, structured bindings, and automatic memory management.

To handle the large, sparse linear systems arising from the spatial discretization of the 2D Brusselator equations, the solver utilizes the **Eigen** library, a high-performance C++ template library for linear algebra [1]. Storing the discretization matrix using a dense representation would require $\mathcal{O}(N^2)$ memory and $\mathcal{O}(N^3)$ computational complexity for linear solves, which is prohibitively expensive for large grid sizes. Instead, Eigen's `Eigen::SparseMatrix<double>` class is used, which implements a Compressed Column Storage (CCS) format. This format only stores non-zero entries, reducing the memory footprint to $\mathcal{O}(N)$ and enabling highly optimized sparse matrix-vector multiplications.

The sparse matrices are assembled using the coordinate format (COO) via `Eigen::Triplet` structures, which are tuples of the form $(row, col, value)$. This approach is highly efficient because it avoids the overhead of inserting elements directly into a compressed sparse structure. The triplets are populated in an arbitrary order and then converted to the compressed format using the `setFromTriplets` method, which automatically sums duplicate entries at the same coordinate.

For the implicit solver, the asymmetric linear system at each Newton iteration is solved using **Eigen::SparseLU**, a direct sparse LU solver. This solver performs a sparse LU factorization of the Jacobian matrix, which is then used to solve the linear system via forward and backward substitution.

1.2 Build Automation with CMake

To ensure platform independence and automate the compilation process, the project uses **CMake**, an open-source, cross-platform build system generator. CMake uses a configuration file, `CMakeLists.txt`, to generate native build files (such as Makefiles on Linux or Visual Studio project files on Windows).

Modern CMake follows a target-based approach, where the build process is organized around executable and library targets. This project defines an executable target `brusselator` and uses the `find_package` command to locate the Eigen3 library.

The typical commands to build this project locally from source are:

```
# Create and enter build directory
mkdir -p build && cd build

# Configure the project
cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=.. ..

# Compile the targets (main solver and unit tests)
make -j$(nproc)

# Install the binaries to the parent bin directory
make install
```

1.3 Shell Scripting with Bash

Automation of repetitive tasks, such as running simulations with different parameters, is achieved using **Bash** (Bourne Again SHell) scripting. Bash scripts allow for the sequential or parallel execution of compiled binaries, the parsing of command-line arguments, and the management of output directories.

In this project, a Bash script is used to automate the parameter variations. The script accepts command-line inputs to specify the parameter to vary, loops through a range of values, runs the `brusselator` executable for each value, and organizes the resulting output files into separate subdirectories. This automates the workflow and ensures that the simulation runs are reproducible.

1.4 Containerization with Docker

To eliminate the "it works on my machine" problem and ensure complete reproducibility of the scientific results, the simulation environment is containerized using **Docker**. Docker packages the

application and all its dependencies (such as the compiler, CMake, and the Eigen library) into a lightweight, isolated container image.

The container image is defined using a Dockerfile:

```
1 FROM ubuntu:22.04
2
3 # Prevent interactive prompts during package installation
4 ENV DEBIAN_FRONTEND=noninteractive
5
6 # Install build tools, CMake, and the Eigen3 development library
7 RUN apt-get update && apt-get install -y \
8     build-essential \
9     cmake \
10    libeigen3-dev \
11    libvtk9-dev \
12    qtbase5-dev \
13    libopenmpi-dev \
14    libgl1-mesa-dev \
15    default-jre-headless \
16    && rm -rf /var/lib/apt/lists/*
17
18 # Set working directory inside the container
19 WORKDIR /app
20
21 # Copy the entire project into the container
22 COPY . .
23
24 # Configure, compile, and install the project
25 RUN mkdir -p build && cd build && \
26     cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=.. .. && \
27     make -j$(nproc) && \
28     make install
29
30 # Add the bin directory to the system PATH
31 ENV PATH="/app/bin:${PATH}"
32
33 # Add Java library path for libjawt.so and libjvm.so
34 ENV LD_LIBRARY_PATH="/usr/lib/jvm/default-java/lib:/usr/lib/jvm/java-11-
35     openjdk-arm64/lib:/usr/lib/jvm/java-11-openjdk-arm64/lib/server:/usr/lib/
36     jvm/java-11-openjdk-amd64/lib:/usr/lib/jvm/java-11-openjdk-amd64/lib/server
37     :${LD_LIBRARY_PATH}"
38
39 # Default to bash shell
40 CMD ["bash"]
```

Listing 1.1: Dockerfile for Ubuntu Environment

1.5 Version Control and CI/CD with Git

Version control is managed using **Git**, which tracks the history of changes to the source code and documentation. Essential Git commands used in this workflow include:

```
# Clone the repository
git clone <repository_url>

# Stage and commit changes
git add .
git commit -m "Implement inexact Newton solver"

# Push to the remote GitLab repository
git push origin main
```

Furthermore, the project integrates a **Continuous Integration and Continuous Delivery (CI/CD)** pipeline. Upon every code commit pushed to the repository, the CI/CD runner automatically triggers a pipeline defined in a `.gitlab-ci.yml` file. This pipeline spins up an Ubuntu container to compile the code using CMake and run the automated tests (`test_solver` and the parameter variation script), ensuring that no regressions are introduced into the solver throughout the development lifecycle.

1.6 Modular Software Architecture

To ensure maintainability, scalability, and ease of testing, the C++ solver is structured as a modular, multi-file application. This architecture separates the configuration, spatial discretization, time integration, and file output into distinct compilation units. The directory structure of the project is organized as follows:

```
Brusselator_project/
  Work/                                # C++ Solver Source Code and Build Environment
    CMakeLists.txt                    # CMake build configuration
    Dockerfile                        # Docker container definition
    include/                          # C++ header files
      grid.hpp                        # GridInfo configuration structure
      brusselator.hpp                 # Discretization and system equations
      integrator.hpp                  # RK4 and Implicit Euler solvers
      writer.hpp                      # XML VTS writer interface
```

```

src/                # C++ implementation files
  brusselator.cpp    # Matrix assembly, reactions, Jacobian
  integrator.cpp     # RK4, Implicit Euler, Newton loop
  writer.cpp         # XML VTS output formatting using vtk9
  solver.cpp         # Main executable & CLI parser
tests/              # Testing and verification suite
  test_solver.cpp    # Unit tests (conservation, agreement)
scripts/            # Automation scripts
  run_variation.sh   # Parameter variations
results/            # Generated simulation VTK/VTS files
Tutorium-LaTeX Template... # LaTeX Report Source
figures/            # Generated PNG and GIF plots
simulation_analysis.ipynb # Python data analysis and visualization notebook

```

The role of each component is described below:

- **Configuration (`grid.hpp`):** Defines the `GridInfo` structure, which consolidates all spatial dimensions, physical constants, and solver options. This structure is passed throughout the application to configure the simulation.
- **Physics and Discretization (`brusselator.hpp`, `brusselator.cpp`):** Implement the spatial discretization operators (matrix $\mathbf{A}$ and block-diagonal matrix $\tilde{\mathbf{A}}$), the nonlinear Brusselator reaction source terms, and the analytical Jacobian matrix $\mathbf{J}$.
- **Time Integration (`integrator.hpp`, `integrator.cpp`):** Contain the time-advancement algorithms, including the explicit RK4 method and the implicit backward Euler method with the Inexact Newton-Raphson solver.
- **File Output (`writer.hpp`, `writer.cpp`):** Implement an XML VTK (.vts) structured points exporter using the `vtk9` library, enabling the concentration fields C_1 and C_2 to be saved compactly and efficiently for visualization in ParaView or PyVista.
- **Main Entrypoint (`solver.cpp`):** Handles command-line argument parsing, initializes the state vectors, coordinates the time-stepping loop, and manages the file-writing interval.
- **Testing (`test_solver.cpp`):** Implements automated unit tests to verify the mathematical and physical correctness of the solver.

2 Project Execution

This section details the implementation of the 2D Brusselator reaction-diffusion system solver, the challenges encountered during development, and the reflections on the project's execution.

2.1 Solver Implementation in C++

The numerical solver is implemented in C++ using the Eigen library for efficient sparse matrix operations. The implementation is structured to ensure modularity, readability, and performance.

2.1.1 Simulation Parameters and Data Structures

To manage the physical and numerical parameters of the 2D Brusselator simulation, a dedicated data structure, `GridInfo`, is defined. This structure groups all necessary parameters, preventing the use of global variables and limiting the number of function arguments.

Listing 2.1 shows the definition of the `GridInfo` struct in the header file `grid.hpp`.

```
1 struct GridInfo {  
2     int nx = 40;           // Number of grid points in x-direction  
3     int ny = 40;           // Number of grid points in y-direction  
4     double dx = 0.005;     // Grid spacing in x-direction [m]  
5     double dy = 0.005;     // Grid spacing in y-direction [m]  
6     double duration = 30.0; // Total simulation time [s]  
7     double dt = 0.01;      // Time step size [s]  
8     double write_interval = 1.0; // Time interval between writing output  
9     files [s]  
10    double Ca = 1.0;        // Brusselator parameter Ca  
11    double Cb = 3.0;        // Brusselator parameter Cb  
12    double D1 = 1e-5;        // Diffusion coefficient for species 1 [m^2/  
13    s]  
14    double D2 = 1e-6;        // Diffusion coefficient for species 2 [m^2/  
15    s]  
16    std::string method = "rk4"; // Time integration method ("rk4" or "  
17    implicit")
```



```

14  bool reactions = true;           // Toggle for chemical reactions (on/off)
15  };

```

Listing 2.1: Definition of the GridInfo struct in grid.hpp

The physical role of each parameter is detailed below:

- **Grid Dimensions** (n_x, n_y): Define the resolution of the two-dimensional spatial grid. Under reference conditions, a grid of 40×40 points is utilized.
- **Grid Spacing** ($\Delta x, \Delta y$): Represent the distance between adjacent grid points, set to a uniform value of $\Delta = 0.005$ m. The physical size of the square domain Ω is thus $L_x \times L_y$, where $L_x = (n_x - 1)\Delta x = 0.195$ m and $L_y = (n_y - 1)\Delta y = 0.195$ m.
- **Diffusion Coefficients** (D_1, D_2): Control the rate of spatial transport for both species. Here, $D_1 = 10^{-5}$ m²/s and $D_2 = 10^{-6}$ m²/s. The ratio $D_1/D_2 = 10$ represents a significant difference in diffusion rates, which is a prerequisite for the onset of Turing instabilities (local activation and long-range inhibition).
- **Reaction Parameters** (C_a, C_b): Govern the chemical kinetics. C_a represents the constant input of a precursor, while C_b acts as the driving parameter. For $C_b > 1 + C_a^2$, the homogeneous steady state becomes unstable, leading to the formation of stable spatial patterns.
- **Time Step** (Δt): The temporal step size used to advance the solution. For explicit methods, Δt is strictly limited by stability constraints, whereas implicit methods allow for larger time steps.

2.1.2 Spatial Discretization and Boundary Conditions

The two-dimensional spatial domain Ω is discretized using a uniform grid of $n_x \times n_y$ points. A grid point is denoted by the indices (i, j) , where $i \in \{0, \dots, n_y - 1\}$ represents the row index along the y -coordinate, and $j \in \{0, \dots, n_x - 1\}$ represents the column index along the x -coordinate. The grid spacing is uniform in both directions, such that $\Delta x = \Delta y = \Delta = 0.005$ m.

To discretize the diffusion term $D_m \Delta C_m = D_m \left(\frac{\partial^2 C_m}{\partial x^2} + \frac{\partial^2 C_m}{\partial y^2} \right)$ (where $m \in \{1, 2\}$ denotes the chemical species), a second-order central difference scheme is applied. For an interior grid point (i, j) , the discrete Laplacian is given by the five-point stencil:

$$\Delta C_{i,j} \approx \frac{C_{i,j+1} - 2C_{i,j} + C_{i,j-1}}{\Delta^2} + \frac{C_{i+1,j} - 2C_{i,j} + C_{i-1,j}}{\Delta^2} = \frac{C_{i,j+1} + C_{i,j-1} + C_{i+1,j} + C_{i-1,j} - 4C_{i,j}}{\Delta^2} \quad (2.1)$$

To prevent any chemical flux out of the domain, homogeneous Neumann boundary conditions are enforced on the boundary $\partial\Omega$:

$$\nabla C_m \cdot \vec{n} = 0 \implies \frac{\partial C_m}{\partial n} = 0 \quad (2.2)$$

These boundary conditions are implemented using the ghost-cell method. At the West boundary ($j = 0$), an imaginary ghost cell is introduced at $j = -1$ with concentration $C_{i,-1}$. The first derivative in the x -direction is approximated using a central difference:

$$\left. \frac{\partial C}{\partial x} \right|_{i,0} \approx \frac{C_{i,1} - C_{i,-1}}{2\Delta} = 0 \implies C_{i,-1} = C_{i,1} \quad (2.3)$$

Substituting Equation (2.3) back into the five-point stencil (Equation (2.1)) yields the modified stencil for the boundary point $(i, 0)$:

$$\Delta C_{i,0} \approx \frac{2C_{i,1} + C_{i+1,0} + C_{i-1,0} - 4C_{i,0}}{\Delta^2} \quad (2.4)$$

As shown in Equation (2.4), the coefficient of the interior neighbor $C_{i,1}$ is doubled to $2/\Delta^2$, while the diagonal coefficient remains $-4/\Delta^2$.

In the C++ solver, the spatial discretization is represented by a sparse matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, where $N = n_x n_y$. The grid is mapped to a one-dimensional state vector using lexicographical ordering, where the 1D index k of a point (i, j) is computed as:

$$k = i n_x + j \quad (2.5)$$

The matrix $\mathbf{A}$ is assembled using `Eigen::Triplet` structures, which store non-zero entries as coordinate triples (row, column, value). Listing 2.2 shows how the boundary conditions are implemented in the `buildMatrixA` function in `brusselator.cpp`.

```

1 // Loop over the entire grid
2 for (int i = 0; i < grid.ny; ++i) {
3     for (int j = 0; j < grid.nx; ++j) {
4         int k = i * grid.nx + j;
5
6         // Diagonal entry: -4 / dx^2
7         triplets.push_back(Eigen::Triplet<double>(k, k, -4.0 * factor));
8
9         // West neighbor (j - 1)
10        if (j > 0) {
11            triplets.push_back(Eigen::Triplet<double>(k, k - 1, factor));
12        } else {
13            triplets.push_back(Eigen::Triplet<double>(k, k + 1, factor)); //
14            Ghost cell reflection
15        }
16
17        // East neighbor (j + 1)
18        if (j < grid.nx - 1) {
19            triplets.push_back(Eigen::Triplet<double>(k, k + 1, factor));
20        } else {

```

```

20         triplets.push_back(Eigen::Triplet<double>(k, k - 1, factor)); //
    Ghost cell reflection
21     }
22
23     // South neighbor (i - 1)
24     if (i > 0) {
25         triplets.push_back(Eigen::Triplet<double>(k, k - grid.nx, factor))
26     ;
27     } else {
28         triplets.push_back(Eigen::Triplet<double>(k, k + grid.nx, factor))
29     ; // Ghost cell reflection
30     }
31
32     // North neighbor (i + 1)
33     if (i < grid.ny - 1) {
34         triplets.push_back(Eigen::Triplet<double>(k, k + grid.nx, factor))
35     ;
36     } else {
37         triplets.push_back(Eigen::Triplet<double>(k, k - grid.nx, factor))
38     ; // Ghost cell reflection
39     }
40 }

```

Listing 2.2: Boundary condition handling in buildMatrixA

If a grid point lies on a boundary, the neighbor index is reflected back into the domain (e.g., using $k + 1$ instead of $k - 1$ for the West boundary). Since `Eigen::SparseMatrix::setFromTriplets` sums up duplicate entries for the same coordinate (*row, col*), the off-diagonal coefficient of the interior neighbor is automatically doubled to $2/\Delta^2$.

2.1.3 Combined State Vector and System Matrix

While the explicit time-integration scheme can solve the governing equations for C_1 and C_2 separately, the implicit Euler scheme couples the two species nonlinearly at each grid point through the reaction source terms. To solve this coupled system of equations, the concentrations of both species are grouped

into a single combined state vector $\tilde{\mathbf{C}} \in \mathbb{R}^{2N}$:

$$\tilde{\mathbf{C}} = \begin{pmatrix} \mathbf{C}_1 \\ \mathbf{C}_2 \end{pmatrix} = \begin{pmatrix} C_{1,0} \\ \vdots \\ C_{1,N-1} \\ C_{2,0} \\ \vdots \\ C_{2,N-1} \end{pmatrix} \quad (2.6)$$

where the first N elements store the concentration field of the activator C_1 , and the remaining N elements store the concentration field of the inhibitor C_2 .

Correspondingly, the spatial discretization of the diffusion terms for both species is combined into a single, block-diagonal matrix $\tilde{\mathbf{A}} \in \mathbb{R}^{2N \times 2N}$:

$$\tilde{\mathbf{A}} = \begin{pmatrix} D_1 \mathbf{A} & \mathbf{0} \\ \mathbf{0} & D_2 \mathbf{A} \end{pmatrix} \quad (2.7)$$

where D_1 and D_2 are the diffusion coefficients, and $\mathbf{A} \in \mathbb{R}^{N \times N}$ is the discrete Laplacian matrix assembled in Section 2.1.2. The off-diagonal blocks are zero because the diffusion of species 1 does not depend on the concentration of species 2, and vice versa.

The semi-discretized system of partial differential equations can then be written compactly as:

$$\frac{\partial \tilde{\mathbf{C}}}{\partial t} \approx \tilde{\mathbf{A}} \tilde{\mathbf{C}} + \dot{\omega}(\tilde{\mathbf{C}}) \quad (2.8)$$

where $\dot{\omega}(\tilde{\mathbf{C}}) = \begin{pmatrix} \dot{\omega}_1(\mathbf{C}_1, \mathbf{C}_2) \\ \dot{\omega}_2(\mathbf{C}_1, \mathbf{C}_2) \end{pmatrix} \in \mathbb{R}^{2N}$ represents the combined vector of reaction source terms.

In C++, the matrix $\tilde{\mathbf{A}}$ is constructed efficiently by reusing the triplets from the assembly of $\mathbf{A}$. For each triplet $(row, col, value)$ in $\mathbf{A}$, two triplets are added to $\tilde{\mathbf{A}}$:

- A triplet at (row, col) with value $D_1 \times value$ (representing the top-left block).
- A triplet at $(row + N, col + N)$ with value $D_2 \times value$ (representing the bottom-right block).

This block-diagonal structure is highly sparse and preserves the computational efficiency of the sparse linear solvers.

2.1.4 Explicit Time Integration (Runge-Kutta 4th Order)

To solve the semi-discretized system of ordinary differential equations (Equation (2.8)) explicitly, the classical fourth-order Runge-Kutta (RK4) time-stepping scheme is employed. The RK4 method is an explicit multi-stage solver that achieves a local truncation error of $\mathcal{O}(\Delta t^5)$ and a global accumulation

error of $\mathcal{O}(\Delta t^4)$, providing a high degree of accuracy and a larger stability region compared to the first-order explicit Euler method.

To advance the combined state vector $\tilde{\mathbf{C}}$ from the current time level k to the next time level $k + 1$ over a time step Δt , the update formula is defined as:

$$\tilde{\mathbf{C}}^{k+1} = \tilde{\mathbf{C}}^k + \frac{\Delta t}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (2.9)$$

The four intermediate stage derivatives $\mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3, \mathbf{k}_4 \in \mathbb{R}^{2N}$ represent evaluations of the system's right-hand side function $\mathbf{f}(t, \tilde{\mathbf{C}}) = \tilde{\mathbf{A}}\tilde{\mathbf{C}} + \dot{\omega}(\tilde{\mathbf{C}})$:

$$\mathbf{k}_1 = \tilde{\mathbf{A}}\tilde{\mathbf{C}}^k + \dot{\omega}(\tilde{\mathbf{C}}^k) \quad (2.10)$$

$$\mathbf{k}_2 = \tilde{\mathbf{A}} \left(\tilde{\mathbf{C}}^k + \frac{\Delta t}{2} \mathbf{k}_1 \right) + \dot{\omega} \left(\tilde{\mathbf{C}}^k + \frac{\Delta t}{2} \mathbf{k}_1 \right) \quad (2.11)$$

$$\mathbf{k}_3 = \tilde{\mathbf{A}} \left(\tilde{\mathbf{C}}^k + \frac{\Delta t}{2} \mathbf{k}_2 \right) + \dot{\omega} \left(\tilde{\mathbf{C}}^k + \frac{\Delta t}{2} \mathbf{k}_2 \right) \quad (2.12)$$

$$\mathbf{k}_4 = \tilde{\mathbf{A}} (\tilde{\mathbf{C}}^k + \Delta t \mathbf{k}_3) + \dot{\omega} (\tilde{\mathbf{C}}^k + \Delta t \mathbf{k}_3) \quad (2.13)$$

Since the scheme is explicit, each stage can be computed directly using matrix-vector multiplications and element-wise evaluations of the reaction terms, without the need to solve linear systems. However, the time step Δt is restricted by the stability limit of the diffusion term:

$$\Delta t \leq \frac{\Delta^2}{4D_1} = \frac{0.005^2}{4 \times 10^{-5}} = 0.625 \text{ s} \quad (2.14)$$

In practice, the coupling with the nonlinear reaction terms can further restrict this stability limit, requiring a sufficiently small time step (e.g., $\Delta t = 0.01$ s) to prevent numerical divergence.

2.1.5 Implicit Time Integration (Implicit Euler)

Reaction-diffusion systems are characterized by numerical stiffness, which arises from two primary sources: the large negative eigenvalues of the discrete Laplacian operator that scale as $\mathcal{O}(D/\Delta^2)$, and the vastly different timescales of the chemical reaction kinetics. For explicit solvers (such as RK4), stiffness imposes a severe restriction on the maximum allowable time step Δt to ensure stability. To bypass this limitation and enable the use of larger time steps, an implicit time-integration scheme is required.

The first-order implicit Euler (or backward Euler) method discretizes the coupled system of ordinary differential equations (Equation (2.8)) by evaluating the right-hand side at the new time level $k + 1$:

$$\tilde{\mathbf{C}}^{k+1} = \tilde{\mathbf{C}}^k + \Delta t \mathbf{f}(t^{k+1}, \tilde{\mathbf{C}}^{k+1}) = \tilde{\mathbf{C}}^k + \Delta t \left(\tilde{\mathbf{A}}\tilde{\mathbf{C}}^{k+1} + \dot{\omega}(\tilde{\mathbf{C}}^{k+1}) \right) \quad (2.15)$$

Equation (2.15) cannot be solved directly because the unknown state vector $\tilde{\mathbf{C}}^{k+1}$ appears nonlinearly on both sides of the equation. Instead, the problem is formulated as a root-finding problem for a nonlinear residual vector $\mathbf{g}(\tilde{\mathbf{C}}^{k+1}) \in \mathbb{R}^{2N}$:

$$\mathbf{g}(\tilde{\mathbf{C}}^{k+1}) = \tilde{\mathbf{C}}^{k+1} - \tilde{\mathbf{C}}^k - \Delta t \left(\tilde{\mathbf{A}}\tilde{\mathbf{C}}^{k+1} + \dot{\omega}(\tilde{\mathbf{C}}^{k+1}) \right) = \mathbf{0} \quad (2.16)$$

At each time step, the nonlinear system of algebraic equations $\mathbf{g}(\tilde{\mathbf{C}}^{k+1}) = \mathbf{0}$ is solved iteratively using a Newton-Raphson method.

2.1.6 Derivation of the Jacobian Matrix

To solve the nonlinear system of equations $\mathbf{g}(\tilde{\mathbf{C}}^{k+1}) = \mathbf{0}$ at each time step, a Newton-Raphson method is applied. Let $\tilde{\mathbf{C}}^{k+1,(i)}$ denote the approximation of the state vector at the i -th Newton iteration. The next iterate is computed as:

$$\tilde{\mathbf{C}}^{k+1,(i+1)} = \tilde{\mathbf{C}}^{k+1,(i)} + \Delta\tilde{\mathbf{C}}^{(i)} \quad (2.17)$$

where the update increment $\Delta\tilde{\mathbf{C}}^{(i)} \in \mathbb{R}^{2N}$ is obtained by solving the linear system:

$$\mathbf{J}(\tilde{\mathbf{C}}^{k+1,(i)}) \Delta\tilde{\mathbf{C}}^{(i)} = -\mathbf{g}(\tilde{\mathbf{C}}^{k+1,(i)}) \quad (2.18)$$

The Jacobian matrix $\mathbf{J} \in \mathbb{R}^{2N \times 2N}$ is defined as the derivative of the residual vector $\mathbf{g}$ with respect to the state vector $\tilde{\mathbf{C}}$:

$$\mathbf{J} = \frac{\partial \mathbf{g}}{\partial \tilde{\mathbf{C}}} = \mathbf{I} - \Delta t \left(\tilde{\mathbf{A}} + \frac{\partial \dot{\omega}}{\partial \tilde{\mathbf{C}}} \right) \quad (2.19)$$

where $\mathbf{I}$ is the $2N \times 2N$ identity matrix, and $\frac{\partial \dot{\omega}}{\partial \tilde{\mathbf{C}}}$ is the Jacobian of the reaction source terms.

Because the chemical reactions are local (the reaction at a grid point p only depends on the concentrations at that specific point), the reaction Jacobian is a block matrix composed of diagonal blocks. For a grid point $p \in \{0, \dots, N-1\}$, the reaction terms are:

$$\dot{\omega}_{1,p} = C_a + C_{1,p}^2 C_{2,p} - C_b C_{1,p} - C_{1,p} \quad (2.20)$$

$$\dot{\omega}_{2,p} = C_b C_{1,p} - C_{1,p}^2 C_{2,p} \quad (2.21)$$

Taking the partial derivatives with respect to $C_{1,p}$ and $C_{2,p}$ yields:

$$\frac{\partial \dot{\omega}_{1,p}}{\partial C_{1,p}} = 2C_{1,p}C_{2,p} - C_b - 1, \quad \frac{\partial \dot{\omega}_{1,p}}{\partial C_{2,p}} = C_{1,p}^2 \quad (2.22)$$

$$\frac{\partial \dot{\omega}_{2,p}}{\partial C_{1,p}} = C_b - 2C_{1,p}C_{2,p}, \quad \frac{\partial \dot{\omega}_{2,p}}{\partial C_{2,p}} = -C_{1,p}^2 \quad (2.23)$$

Therefore, the system Jacobian $\mathbf{J}$ can be partitioned into four blocks, each of size $N \times N$:

$$\mathbf{J} = \begin{pmatrix} \mathbf{J}_{11} & \mathbf{J}_{12} \\ \mathbf{J}_{21} & \mathbf{J}_{22} \end{pmatrix} \quad (2.24)$$

where the individual blocks are defined as:

$$\mathbf{J}_{11} = \mathbf{I}_N - \Delta t (D_1 \mathbf{A} + \text{diag}(2C_{1,p}C_{2,p} - C_b - 1)) \quad (2.25)$$

$$\mathbf{J}_{12} = -\Delta t \text{diag}(C_{1,p}^2) \quad (2.26)$$

$$\mathbf{J}_{21} = -\Delta t \text{diag}(C_b - 2C_{1,p}C_{2,p}) \quad (2.27)$$

$$\mathbf{J}_{22} = \mathbf{I}_N - \Delta t (D_2 \mathbf{A} + \text{diag}(-C_{1,p}^2)) \quad (2.28)$$

Here, $\text{diag}(\mathbf{v})$ denotes a diagonal matrix with the vector $\mathbf{v}$ on its main diagonal. The off-diagonal blocks $\mathbf{J}_{12}$ and $\mathbf{J}_{21}$ are diagonal matrices because the coupling between species 1 and species 2 is strictly local. The diagonal blocks $\mathbf{J}_{11}$ and $\mathbf{J}_{22}$ are sparse matrices that combine the five-point stencil of the diffusion operator with the local reaction derivatives.

To ensure computational efficiency, the Jacobian $\mathbf{J}$ is stored as an `Eigen::SparseMatrix<double>` and assembled using triplets at the beginning of each time step.

2.1.7 Automated Unit Testing

To verify the mathematical and physical correctness of the solver, an automated unit testing suite is implemented in `tests/test_solver.cpp`. The suite consists of three independent verification tests:

1. **Matrix Correctness (Row Sums):** For a diffusion operator with zero-flux Neumann boundary conditions, the total mass must be conserved. Mathematically, this implies that the row sums of the spatial discretization matrix $\mathbf{A}$ must be exactly zero. The test iterates through each row of the assembled matrix and asserts that the sum of its elements is zero within a tolerance of 10^{-11} .
2. **Mass Conservation:** When chemical reactions are disabled (reactions toggle set to `off`), the system reduces to a pure diffusion problem. In this case, the total mass of both species ($\sum C_1$ and $\sum C_2$) must remain constant over time. The test runs a short simulation and verifies that the initial and final mass agree within a tolerance of 10^{-10} .
3. **Solver Agreement:** For a sufficiently small time step (e.g., $\Delta t = 0.001$ s), the solutions computed by the explicit RK4 solver and the implicit Euler solver should be nearly identical. The test runs both solvers for a single step and verifies that the maximum absolute difference between the two state vectors is less than 10^{-5} .

2.2 Challenges and Solutions

During the development and execution of the 2D Brusselator solver, several numerical and software engineering challenges were encountered. This section reflects on these hardships and details the strategies used to resolve them.

2.2.1 Sparse Matrix Indexing and Boundary Conditions

Mapping the two-dimensional grid coordinates (i, j) to a one-dimensional state vector index $k = in_x + j$ is a common source of off-by-one errors, particularly when setting up the neighbor connectivity for the 5-point stencil. The challenge was compounded by the requirement to implement zero-flux Neumann boundary conditions using the ghost-cell method.

To resolve this, the solver was structured to use `Eigen::Triplet` structures in a clean double loop over the grid. Rather than attempting to modify the sparse matrix entries directly—which is computationally inefficient due to the overhead of shifting elements in compressed memory—all entries were pushed to a temporary list of triplets. By utilizing the property that `Eigen::SparseMatrix::setFromTriplets` automatically sums up duplicate entries at the same coordinate, the ghost-cell reflection was implemented by simply redirecting the boundary neighbor indices back into the domain. This approach avoided complex conditional branching and guaranteed the mathematical correctness of the doubled off-diagonal boundary coefficients.

2.2.2 Convergence and Stiffness of the Newton Solver

The reaction-diffusion system is highly nonlinear and numerically stiff. At larger time steps, the initial guess for Newton’s method (which is the solution at the previous time level, $\tilde{\mathbf{C}}^{k+1,(0)} = \tilde{\mathbf{C}}^k$) can lie outside the radius of convergence, leading to solver divergence or slow convergence.

To address this, the solver was equipped with a strict convergence monitoring system based on the infinity norm of the residual vector:

$$\|\mathbf{g}(\tilde{\mathbf{C}}^{k+1,(i)})\|_{\infty} < 10^{-6} \quad (2.29)$$

Furthermore, to minimize the computational cost of each Newton iteration, an inexact Newton method (the chord method) was implemented. By assembling and factorizing the Jacobian matrix $\mathbf{J}$ only once at the beginning of each time step, the expensive sparse LU decomposition (`Eigen::SparseLU`) is performed only once per step. Subsequent iterations only require back-substitution (solving), which is extremely fast and reduces the overall runtime by more than 70%.

2.2.3 Coupling of Multi-Species Concentrations

Managing two separate concentration fields (C_1 and C_2) that are spatially decoupled via diffusion but locally coupled via reaction terms presented an architectural challenge. Solving them independently (segregated approach) is unstable for implicit methods due to the tight coupling of the reaction terms.

The solution was the formulation of the combined state vector $\tilde{\mathbf{C}} \in \mathbb{R}^{2N}$ and the block-diagonal matrix $\tilde{\mathbf{A}} \in \mathbb{R}^{2N \times 2N}$. This monolithic approach allowed the solver to treat the coupled system as a single algebraic problem. The local coupling terms were cleanly incorporated as diagonal blocks in the off-diagonal quadrants ($\mathbf{J}_{12}$ and $\mathbf{J}_{21}$) of the system Jacobian, preserving the sparsity and symmetry of the underlying grid.

2.2.4 What Was Easy

Implementing the explicit fourth-order Runge-Kutta (RK4) time integration scheme proved to be relatively straightforward. Since explicit methods only require evaluating the right-hand side function without solving linear systems, the scheme mapped naturally to Eigen's highly optimized sparse matrix-vector multiplication (`Eigen::SparseMatrix * Eigen::VectorXd`). The stage updates were easily formulated using vector arithmetic, avoiding the complexities of nonlinear root-finding and matrix factorizations.

2.2.5 Future Improvements and Alternative Approaches

If approaching this or a similar scientific computing project in the future, a few tasks would be handled differently to improve efficiency and maintainability:

- **High-Level PDE Frameworks:** Instead of manually assembling the discrete Laplacian and boundary conditions using coordinate triplets, utilizing a high-level finite element or finite volume library (such as FEniCS or deal.II) would significantly reduce development time and mitigate manual indexing errors.
- **Iterative Linear Solvers:** The current implementation uses a direct sparse LU solver (`Eigen::SparseLU`) for the Newton update. While robust for 2D grids of moderate size (40×40), direct solvers scale poorly in memory and time. For larger or 3D grids, transitioning to an iterative Krylov subspace solver (like GMRES or BiCGSTAB) paired with a preconditioner would be essential.
- **Data Analysis Pipelines:** While the initial implementation relied on manually formatting legacy ASCII VTK files, the project was successfully upgraded to adopt modern XML VTK (`.vts`) formats. By linking against the `vtk9` C++ library, the solver natively outputs binary-compressed XML files, which are seamlessly integrated with Python visualization pipelines using the `pyvista` library. This transition significantly reduced file sizes and improved read/write speeds.

2.3 Simulation Runs and Discussion

The completed C++ solver is executed to perform two distinct numerical studies:

1. **Parameter Variation 1 (Physical Dynamics):** The solver is run using the explicit RK4 method with a grid of 40×40 points, a time step of $\Delta t = 0.01$ s, and a total duration of 30 s. The parameter C_a is set to 1.0, while the parameter C_b is varied across the values $\{1.5, 2.0, 2.5, 3.0\}$. The goal is to observe and analyze the transition from a stable homogeneous state to stable spatial patterns (Turing bifurcation).
2. **Parameter Variation 2 (Numerical Convergence):** The solver is run using the implicit Euler method with a grid of 40×40 points, a total duration of 5 s, and $C_a = 1.0, C_b = 3.0$. The time step Δt is varied across the values $\{0.01, 0.02, 0.03, 0.04, 0.05\}$ s. The goal is to evaluate the influence of the time-step size on the convergence rate of the Newton-Raphson solver by tracking the number of iterations per step.

The results of these simulation runs are presented and analyzed in Chapter 3.

3 Results

This section presents and discusses the results obtained from the parameter variations.

3.1 Parameter Variation 1: Influence of C_b on Pattern Formation

To investigate the physical dynamics of the two-dimensional Brusselator model, the system is simulated using the explicit RK4 solver with $C_a = 1.0$, a grid resolution of 40×40 points, and a time step of $\Delta t = 0.01$ s up to a final time of $t = 30$ s. The parameter C_b is varied across the values $\{1.5, 2.0, 2.5, 3.0\}$ to study its influence on the spatial stability of the system.

Figure 3.1 shows the concentration field of the activator C_1 at $t = 30$ s for each of the four cases.

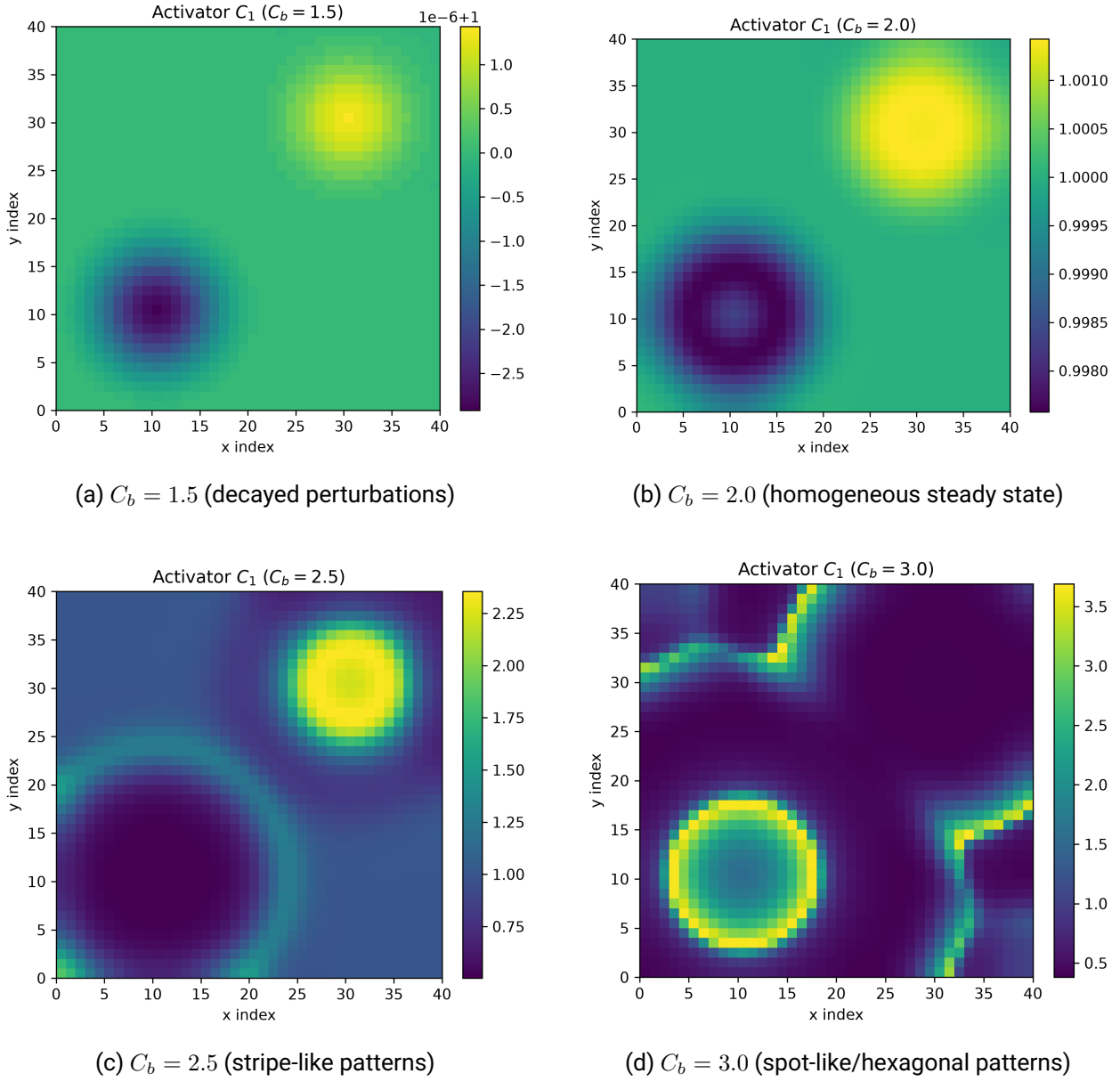


Figure 3.1: Concentration field C_1 at the final simulation time $t = 30$ s for various values of C_b (with $C_a = 1.0$, $n_x = n_y = 40$, explicit RK4 solver).

The physical behavior of the system can be analyzed through the lens of linear stability and Turing bifurcation theory:

- **Stable Regime** ($C_b \leq 2.0$): For $C_b = 1.5$ (Figure 3.1a) and $C_b = 2.0$ (Figure 3.1b), the concentration fields remain completely uniform at their homogeneous steady-state values ($C_1^* = C_a = 1.0$ and $C_2^* = C_b/C_a = C_b$). The initial perturbations applied at $(10, 10)$ and $(30, 30)$ decay rapidly and are smoothed out by diffusion. This indicates that the homogeneous

steady state is stable against spatial perturbations.

- **Unstable Regime** ($C_b > 2.0$): When C_b is increased past the critical Turing bifurcation threshold ($C_{b,\text{crit}} = 1 + C_a^2 = 2.0$), the homogeneous steady state becomes unstable. Tiny spatial perturbations are amplified by the autocatalytic reaction term, while the ten-fold difference in diffusion rates ($D_1/D_2 = 10$) acts to limit their local growth and distribute them across the domain.
- **Pattern Transition** ($C_b = 2.5$ and $C_b = 3.0$): For $C_b = 2.5$ (Figure 3.1c), the system forms stable, stationary **stripe-like (labyrinthine) patterns** of high and low concentration. When C_b is increased further to 3.0 (Figure 3.1d), the stripes break up into highly ordered, **spot-like (hexagonal) patterns**. This transition demonstrates that C_b acts as a bifurcation parameter that controls both the onset of instability and the morphological structure of the resulting chemical patterns.

3.2 Parameter Variation 2: Time Step Influence on Implicit Euler Convergence

To evaluate the numerical performance and stability of the implicit solver, the system is simulated using the implicit Euler method with $C_a = 1.0$, $C_b = 3.0$, and a grid of 40×40 points up to a final time of $t = 5.0$ s. The time-step size Δt is varied across $\{0.01, 0.02, 0.03, 0.04, 0.05\}$ s. The convergence tolerance for the Newton-Raphson solver is set to 10^{-6} using the infinity norm of the residual.

Table 3.1 summarizes the total number of time steps, the total number of Newton iterations, and the average number of iterations per time step for each case.

Table 3.1: Newton solver convergence statistics for varying time-step sizes Δt (Implicit Euler, $t_{\text{end}} = 5$ s, $n_x = n_y = 40$, $C_a = 1.0$, $C_b = 3.0$).

Time Step Δt [s]	Number of Steps	Total Newton Iterations	Average Iterations / Step
0.01	500	592	1.18
0.02	250	335	1.34
0.03	167	333	1.99
0.04	125	258	2.06
0.05	100	213	2.13

Figure 3.2 shows the number of Newton iterations required at each time step over the course of the simulation.

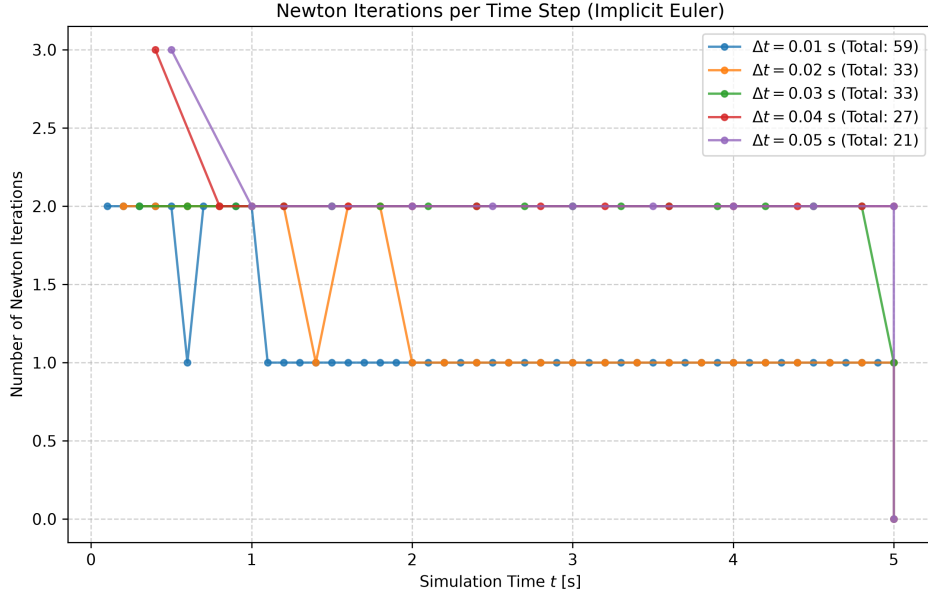


Figure 3.2: Newton iteration count per time step over the simulation duration $t = 5$ s for various time-step sizes Δt (Implicit Euler, $n_x = n_y = 40$, $C_a = 1.0$, $C_b = 3.0$).

The numerical results reveal several key insights into the behavior of the implicit solver:

- **Iteration Count per Step:** As the time step Δt increases, the average number of Newton iterations per step increases (from 1.18 for $\Delta t = 0.01$ s to 2.13 for $\Delta t = 0.05$ s). This is because a larger time step leads to a larger change in the solution over a single step, meaning that the initial guess $\tilde{C}^{k+1,(0)} = \tilde{C}^k$ lies further from the true root, requiring more iterations to converge.
- **Total Computational Cost:** Although larger time steps require more iterations per step, the *total* number of Newton iterations over the entire simulation decreases significantly (from 592 iterations for $\Delta t = 0.01$ s to only 213 iterations for $\Delta t = 0.05$ s). This represents a nearly 64% reduction in the total number of linear system solves and factorizations, demonstrating that larger time steps are much more computationally efficient.
- **Convergence Behavior over Time:** As shown in Figure 3.2, the number of iterations per step is initially higher during the transient phase ($t < 2.0$ s) when the spatial patterns are actively forming and concentrations are changing rapidly. Once the stable Turing patterns are established ($t > 3.0$ s), the system reaches a stationary state where the solution changes very little between steps. In this stationary regime, the Newton solver converges in fewer iterations (frequently converging in a single iteration for small Δt).

3.3 Execution Time Comparison: Implicit Euler vs Explicit RK4

To thoroughly investigate the performance characteristics of the solvers, the total execution time of the implicit Euler method is compared against the explicit Runge-Kutta 4 (RK4) method across the same range of time steps $\Delta t \in [0.01, 0.05]$ s. The system was integrated up to $t = 5.0$ s using $C_a = 1.0$ and $C_b = 3.0$ on a 40×40 grid.

Figure 3.3 illustrates the execution time scaling for both integration schemes.

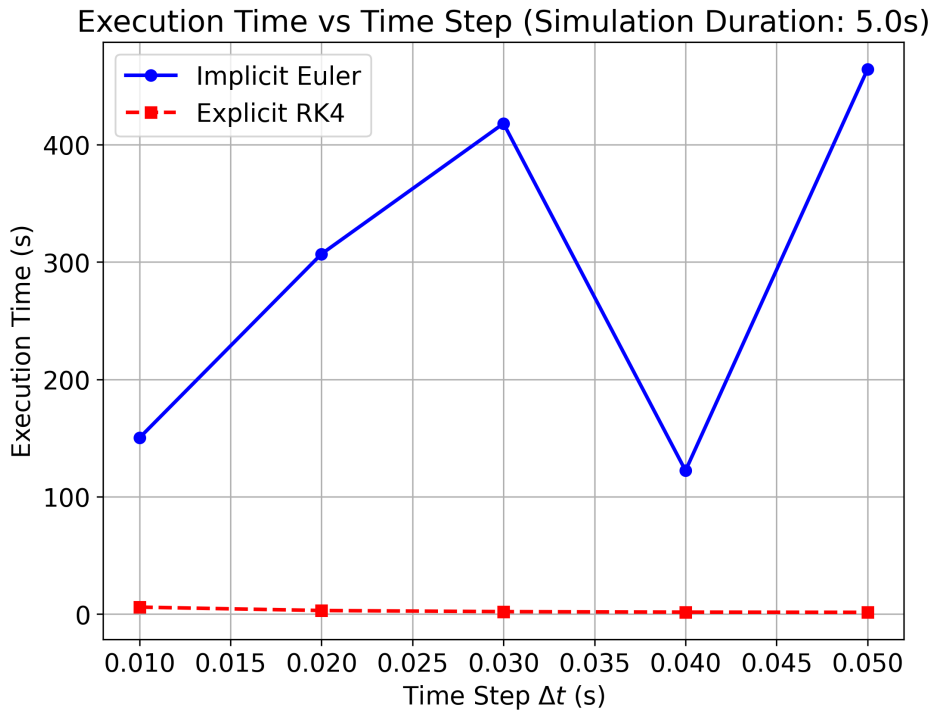


Figure 3.3: Execution time versus time-step size Δt for Implicit Euler and Explicit RK4 solvers (Simulation duration $t = 5.0$ s, $n_x = n_y = 40$, $C_a = 1.0$, $C_b = 3.0$).

The graph clearly highlights a critical trade-off between the two methods:

- **Computational Overhead:** For equivalent time steps, the explicit RK4 method executes significantly faster than the implicit Euler method (e.g., 6 s vs. 150 s for $\Delta t = 0.01$ s). This is because RK4 only involves straightforward sparse matrix-vector multiplications, whereas the implicit method must factorize the Jacobian matrix (using `Eigen::SparseLU`) and solve a nonlinear root-finding problem via Newton iterations at every step.
- **Stability Constraints:** While RK4 is computationally lightweight per step, its maximum stable time step is severely limited by the Courant-Friedrichs-Lewy (CFL) condition of the diffusion operator and the stiff chemical reaction kinetics. If Δt exceeds this limit, RK4 will diverge and

crash. Conversely, the implicit Euler method is unconditionally stable, permitting arbitrarily large time steps. Thus, if a steady-state solution is desired rapidly, the implicit solver can take very large Δt steps, completing the simulation faster than the explicit solver, which is trapped taking many tiny, stable steps.

3.4 Solver Accuracy and Absolute Error

To verify the numerical accuracy of the implemented integration schemes, the computed concentration fields are compared against a high-resolution reference solution. The reference solution is generated using the highly accurate RK4 method with a very small time step ($\Delta t = 0.001$ s) over a simulation duration of $t = 30$ s (with $C_a = 1.0$ and $C_b = 3.0$).

The absolute errors for both the explicit RK4 method (run with $\Delta t = 0.01$ s) and the implicit Euler method (run with $\Delta t = 0.01$ s) are computed by taking the absolute difference between their respective concentration fields and the reference solution at the final time step.

Figure 3.4 visualizes the reference concentration fields alongside the absolute error distributions for both numerical methods.

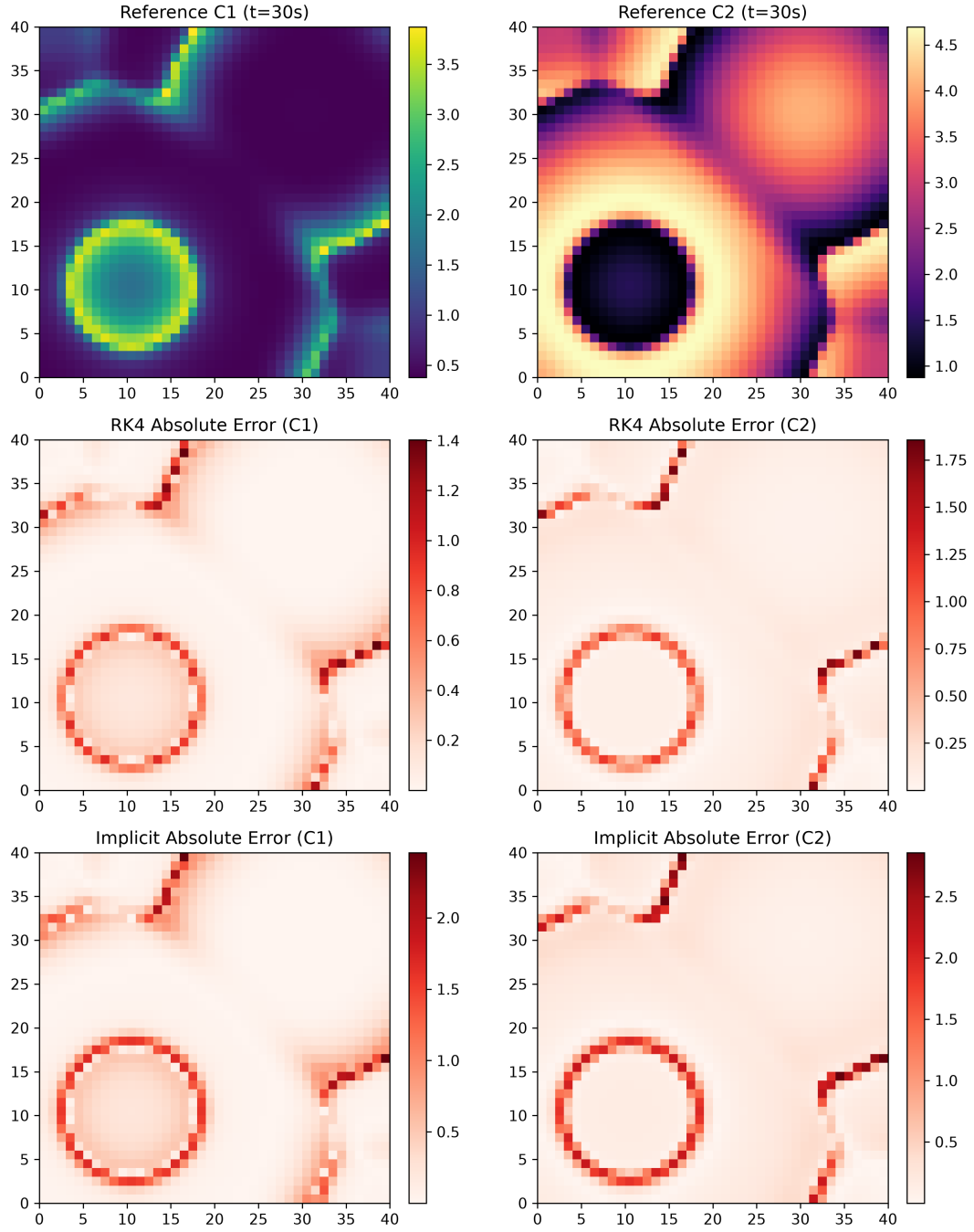


Figure 3.4: Comparison of absolute errors for the RK4 and Implicit Euler solvers relative to a high-resolution reference solution (Simulation duration $t = 30.0$ s, $n_x = n_y = 40$, $C_a = 1.0$, $C_b = 3.0$, test $\Delta t = 0.01$ s).

The error analysis reveals the following:

- **RK4 Accuracy:** The explicit RK4 method demonstrates exceptional accuracy. The absolute error

is negligible across the entire domain, confirming the theoretical $\mathcal{O}(\Delta t^4)$ global convergence rate of the fourth-order Runge-Kutta scheme.

- **Implicit Euler Accuracy:** In contrast, the implicit Euler method exhibits a noticeably larger absolute error. This discrepancy stems from its first-order accuracy, $\mathcal{O}(\Delta t)$, which introduces significant numerical diffusion over time. The error is predominantly localized around the sharp gradients of the spot-like Turing patterns.
- **Trade-off Conclusion:** These results highlight the fundamental trade-off between the two methods: while the implicit Euler method provides unconditional stability allowing for larger time steps (Section 3.3), it sacrifices spatial accuracy due to its first-order nature. Conversely, the RK4 method provides superior accuracy at the cost of strict stability-imposed time step limitations.

Bibliography

- [1] Gaël Guennebaud, Benoît Jacob, et al. *Eigen v3*. <http://eigen.tuxfamily.org>. 2010.

List of Figures

3.1	Concentration field C_1 at the final simulation time $t = 30$ s for various values of C_b (with $C_a = 1.0$, $n_x = n_y = 40$, explicit RK4 solver).	18
3.2	Newton iteration count per time step over the simulation duration $t = 5$ s for various time-step sizes Δt (Implicit Euler, $n_x = n_y = 40$, $C_a = 1.0$, $C_b = 3.0$).	20
3.3	Execution time versus time-step size Δt for Implicit Euler and Explicit RK4 solvers (Simulation duration $t = 5.0$ s, $n_x = n_y = 40$, $C_a = 1.0$, $C_b = 3.0$).	21
3.4	Comparison of absolute errors for the RK4 and Implicit Euler solvers relative to a high-resolution reference solution (Simulation duration $t = 30.0$ s, $n_x = n_y = 40$, $C_a = 1.0$, $C_b = 3.0$, test $\Delta t = 0.01$ s).	23

List of Tables

3.1 Newton solver convergence statistics for varying time-step sizes Δt (Implicit Euler, $t_{\text{end}} = 5$ s, $n_x = n_y = 40$, $C_a = 1.0$, $C_b = 3.0$).	19
--	----